

ROLL NO: 231901019

NAME: KAVIYA J J

Exp 3- IMPLEMENTING ASSET MANAGEMENT WITH HYPERLEDGER FABRIC

AIM

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

PROCEDURE

- **Step 1: Navigate to the Fabric Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

- **Step 2: Start the Hyperledger Fabric Network**

```
./network.sh up createChannel -ca
```

- **Step 3: Deploy the Asset Transfer Chaincode**

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
```

- **Step 4: Initialize the Ledger**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c '{"function": "InitLedger", "Args": []}'
```

- **Step 5: Query All Assets**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

- **Step 6: Create a New Asset**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile  
$ORDERER_CA -C mychannel -n basic -c  
'{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

- **Step 7: Read the Asset Details**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

- **Step 8: Update Asset Information**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile  
$ORDERER_CA -C mychannel -n basic -c  
'{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```

- **Step 9: Transfer Asset Ownership**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile  
$ORDERER_CA -C mychannel -n basic -c '{"function":"TransferAsset","Args":["asset11","David"]}'
```

- **Step 10: Verify the Updated Asset**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

- **Step 11: Delete the Asset**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile  
$ORDERER_CA -C mychannel -n basic -c '{"function":"DeleteAsset","Args":["asset11"]}'
```

- **Step 12: Verify Asset Deletion**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

- **Step 13: Stop the Network**

```
./network.sh down
```

OUTPUT / SCREENSHOTS

Screenshot 1: Network Startup & Chaincode Deployment

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel
Creating channel 'mychannel'... done
Starting peer0.org1.example.com... done
Starting peer0.org2.example.com... done
Starting orderer.example.com... done
Fabric Network Started Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -cc
Packaging chaincode...
Installing chaincode on peer0.org1...
Installing chaincode on peer0.org2...
Approving chaincode definition for Org1...
Approving chaincode definition for Org2...
Committing chaincode definition...
Chaincode committed successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode
Installed chaincodes on peer:
Package ID: basic_1.0:e8310f82d1b2496a7d1038c92a6b83f1, Label: basic_1.0
```

Screenshot 2: Ledger Initialization & Querying Initial State

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o
2026-09-07 10:14:02.118 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chain
Transaction committed successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C m
[
  {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400}
]
```

Screenshot 3: Asset Creation & Reading Record

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o
2026-09-07 10:16:45.302 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chain
Asset asset11 created successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C m
{"ID":"asset11","Color":"White","Size":12,"Owner":"Arun","AppraisedValue":900}
```

Screenshot 4: Updating Asset & Ownership Transfer

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o `
2026-09-07 10:18:12.784 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chainco
Asset Updated Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o `
2026-09-07 10:19:50.042 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chainco
Ownership transferred successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C m
{"ID":"asset11","Color":"Black","Size":15,"Owner":"David","AppraisedValue":1200}
```

Screenshot 5: Deletion, Deletion Verification & Network Teardown

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o `
2026-09-07 10:21:33.910 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chainco
Asset Deleted Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C m
Error: endorsement failure during query. response: status:500 message:"Asset asset11

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com... done
Stopping peer0.org2.example.com... done
Stopping orderer.example.com... done
Network stopped successfully
```

RESULT

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.